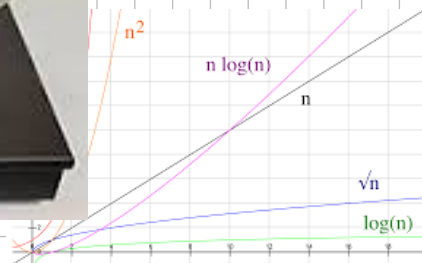
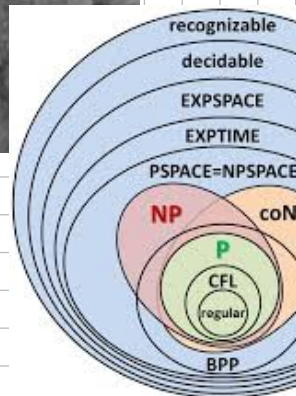
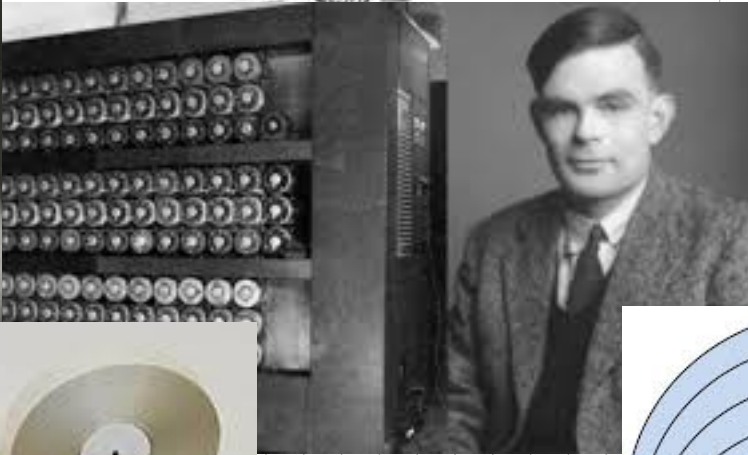
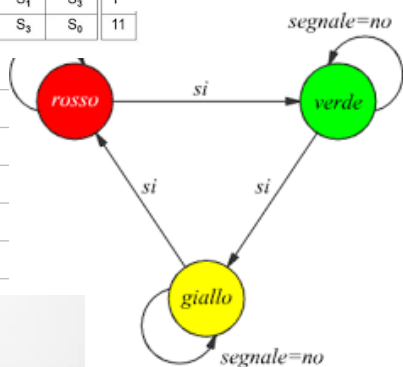
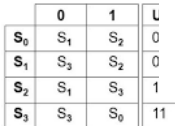
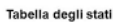
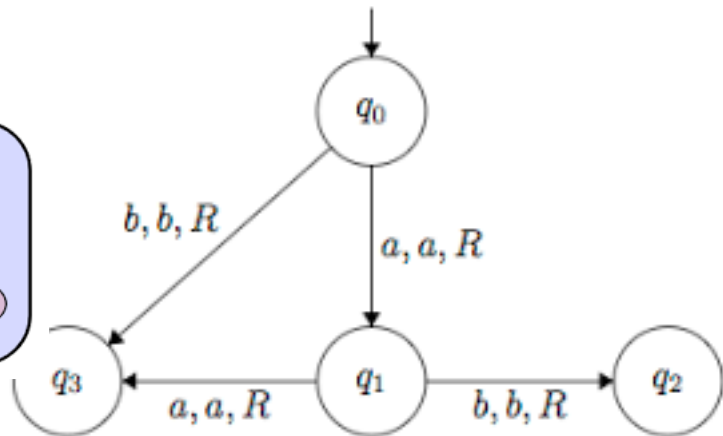


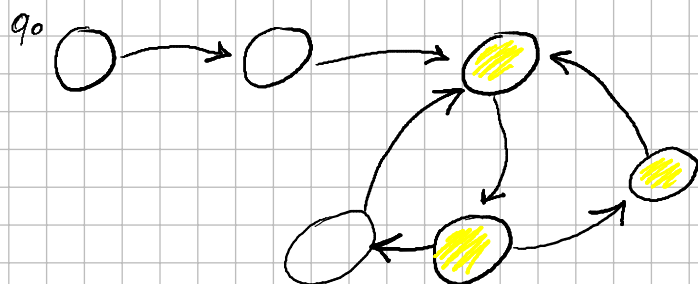
<https://github.com/Exyss/university-notes/blob/main/Bachelor/Terzo%20Anno/Automi%2C%20Calcolabilita%20e%20Complessita.pdf>



LINGUAGGI REGOLARI

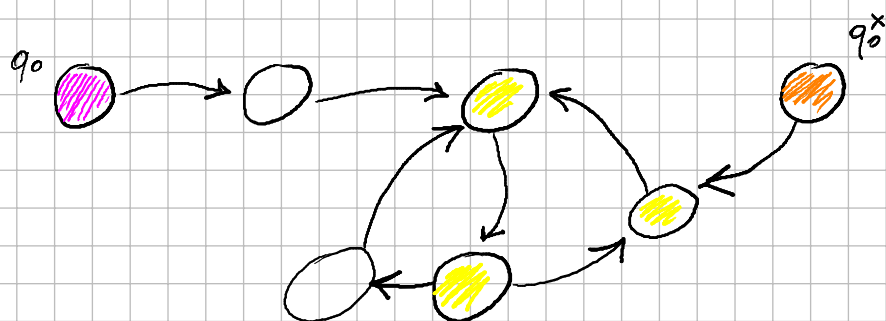
ESERCIZIO 1.1.

PARTIAMO DA UN AUTOMA $\triangleright$ DFA $(Q, \Sigma, \delta, q_0, F)$



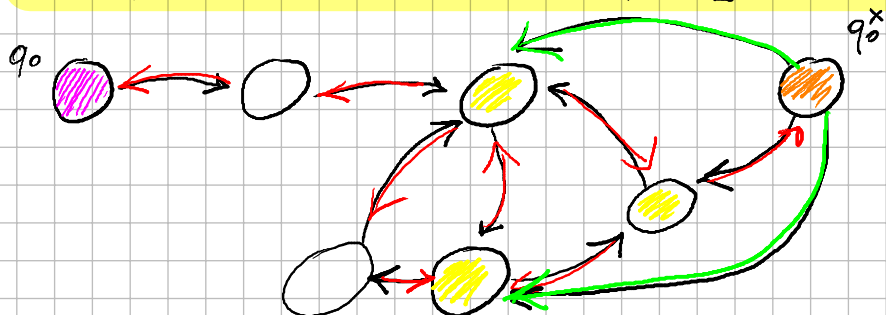
(i nodi in giallo sono i nodi appartenenti a F)

CREIAMO ORA UN NUOVO AUTOMA D_2 NFA



(il nodo nuovo è un nuovo stato, $F_2 = \{q_0\}$)

INVERTO IL VERSO DI TUTTE LE FRECCIE



IN SIMBOLI:

$$D_2 = \text{NFA } (Q_2, \Sigma, \delta_2, q_0^*, F_2 = \{q_0\})$$

$$Q_2 = Q$$

$$\begin{cases} \forall q_1, q_2 \in Q & \delta_2(q_1, a) = q_2 \iff \delta(q_2, a) = q_1 \\ \text{se } a = \epsilon, q_1 \in F \rightarrow \delta_2(q_0^*, \epsilon) = q_1 \end{cases}$$

$$F_2 = \{q_0\}$$

POSSIAMO DIRE CHE $w \in D \iff w \in D_2$

□

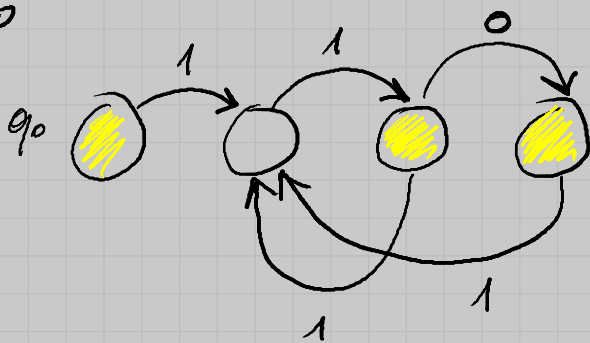
ESERCIZIO 1.2

$$L = \{11, 110\}^*$$

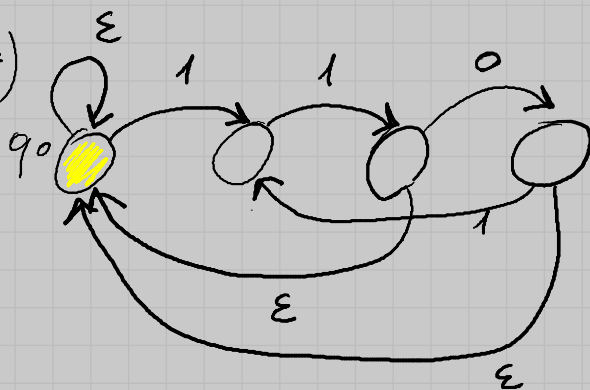
QUESTO E' UN LINGUAGGIO POSTO SOTTO OPERATORE STAR DI KLEENE

$$L^* = \{11, 110\}^* = \epsilon, 11, 110, 11110, 11011, 1111110 \dots$$

DFA DIRETTO



NFA $(Q, \Sigma, \delta, q_0, F)$



DFA $(Q_2, \Sigma_2, \delta_2, q_0^*, F_2)$

IN SIMBOLI:

$$Q_2 = Q$$

$$\delta_2(r, a) = \left(\bigcup_{r \in R} \delta(r, a) \right)$$

$$q_0^* = \epsilon q_0$$

$$F_2 = R \mid (R \cap F) \neq \emptyset$$

ESERCIZIO 1.3.

$$R = (01^+)^*$$

costanza D t.e.

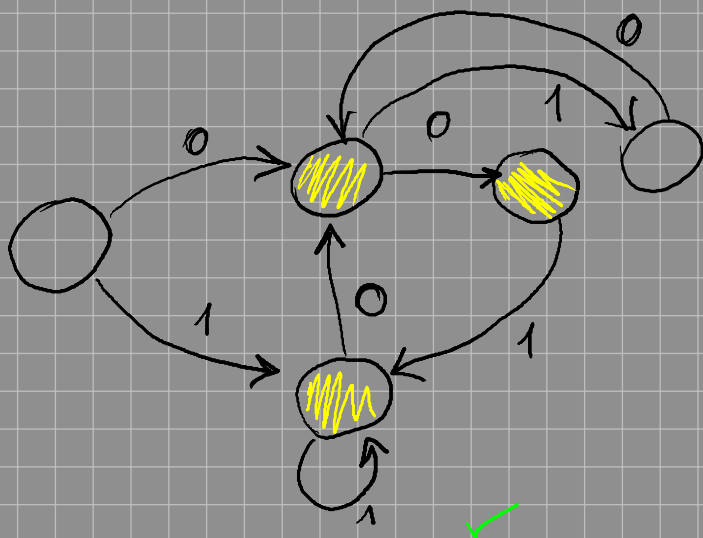
$$L(D) = (w \in \{0,1\}^* \wedge w \notin L(R))$$

$$R = \varepsilon, 01, 0101, 010101, \dots$$

$$R = (01)^*$$

ALLORA

$$L(D) = \varepsilon, 0, 1, 010, 01010, \dots, 101, 111, 1011, \dots$$



ESERCIZIO 1.4.

$$L = \{ w \in \{0,1\}^* \mid |w|_0 = |w|_1 \}$$

DIMOSTRARE CHE $L \notin REG$

DIMOSTRO TRAMITE IL PUMPING LEMMA

- Sia p la lunghezza del pumping
consideriamo la stringa

$$w = 0^p 1^p$$

$$\left. \begin{array}{l} |xy| \leq p \\ |y| \geq 1 \end{array} \right\} w = 0^m 0^{p-m} 1^p = 0^{p-k} 0^k 1^p = 0^{p-k} (0^k)^j 1^p$$

$$p-k \quad k=p-m$$

$$j$$

PROVIAMO PER $j = 0$

$$0^{p-k} 1^p$$

! questa stringa non è in $L \rightarrow L \notin REG$

ESERCIZIO 1.5.

$$L = \{ 1^{n^2} \mid n \in \mathbb{N} \} \in REG? \quad ? \quad ? \quad ?$$

USIAMO IL PUMPING LEMMA

Sopprimiamo che:

$$|xy| \leq P$$

$$|y| \geq 0$$

$$1^{m^2} = \underbrace{1^m \cdot 1^m \cdot 1^m \cdot \dots \cdot 1^m}_{m \text{ VOLTE}}$$

$$\omega = \underbrace{1^P \cdot 1^P \cdot 1^P}_{(P-K) \text{ VOLTE}} \cdot \underbrace{1^P}_{(K \text{ VOLTE})} = \underbrace{1^P}_{(P-M \text{ VOLTE})} \cdot \underbrace{1^P}_{(M \text{ VOLTE})} \cdot \underbrace{1^P}_{(P \text{ VOLTE})}^i$$

PROVIAMO CON $i = 0$

OTTENGO:

$$\underbrace{1^P \cdot 1^P \cdot 1^P \cdot 1^P \cdot \dots \cdot 1^P}_{P-M \text{ VOLTE}} \cdot \underbrace{1^P \cdot \dots \cdot 1^P}_P \notin L \rightarrow L \notin \text{REG}$$

1 Automi

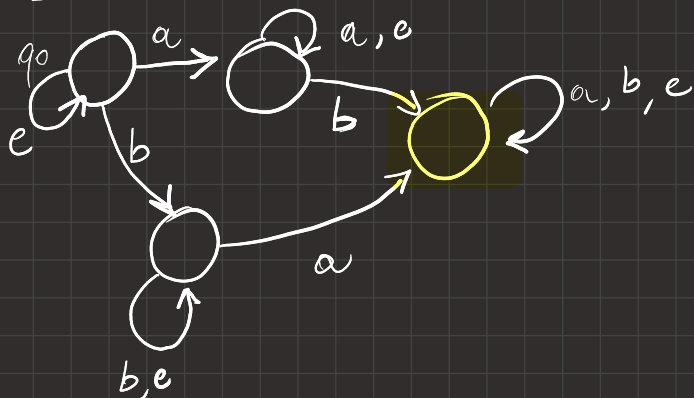
10 Points

Si scriva un'espressione regolare per il linguaggio costituito dall'insieme di stringhe su alfabeto $\{a, b, c\}$ contenenti almeno una a ed almeno una b . Si determini un DFA che riconosca lo stesso linguaggio.

Definire la forma normale di Chomsky per grammatiche acontestuali. Dimostrare che ogni linguaggio acontestuale è generato da una grammatica acontestuale in forma normale di Chomsky.

1) DATO CHE $L(DFA) = L(RE)$

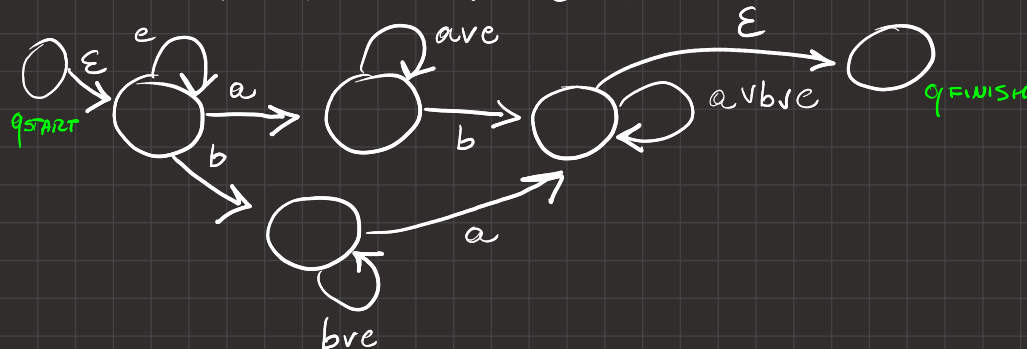
DISEGNAMO PRIMA L'AUTOMA



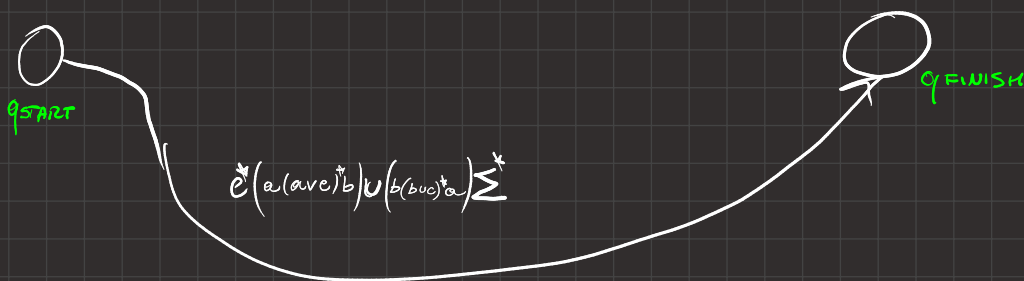
STRINGHE ACCETTATE.

aabe
abe
bea
aeb
ab
aaba ...

LO CONVERTIAMO IN UN GNFA EQUIVALENTE



RIDUCIAMO



ESPRESSIONE: $e^*(a(abc)^*b)u(b(bac)^*a)\Sigma^*$

2) Una grammatica in forma normale di Chomsky è una CFG G tale che: tutte le regole in R di G hanno le forme seguenti:

$S \rightarrow \epsilon, A \rightarrow a, A \rightarrow BC$

dove $S = \text{STATO INIZIALE}$

$A \in V \wedge A \notin \Sigma$

$B, C \in V \cup \Sigma$

$a \in \Sigma$

• VERO PERCHÉ¹ OGNI LINGUAGGIO ACONTESTUALE È GENERATO DA UNA CFG E PER OGNI CFG G ESISTE G' CFG IN CNF.

È POSSIBILE MODIFICARE G PER RENDERLO IN CNF MEDIANTE IL SEGUENTE ALGORITHM.

1. CREO UN NUOVO STATO INIZIALE S_0 , CREO LA REGOLA $S_0 \rightarrow S$
2. SE HO REGOLE TIPO $A \rightarrow \epsilon$ (dette unitarie) LE ELIMINO, POI PER OGNI REGOLA CON A A DESTRA AGGIUNGO REGOLE CON TUTTE LE COMBINAZIONI. ESEMPIO: HO $B \rightarrow \epsilon A \epsilon A \epsilon$
AGGIUNGO $B \rightarrow \epsilon \epsilon A \epsilon$
 $B \rightarrow \epsilon \epsilon \epsilon$
...
3. SE HO REGOLE TIPO $A \rightarrow B$ DOVE $A, B \in V \cup \Sigma$, SE HO REGOLE TIPO $B \rightarrow c$ dove $c \in \Sigma \cup V$, RIMUOVO $B \rightarrow c$, AGGIUNGO $A \rightarrow c$
4. SE HO REGOLE TIPO $A \rightarrow u_1 u_2 \dots u_k$ DOVE $\forall u_i, u_i \in \Sigma$ CREO $A_1, \dots, A_{k-1}$ DOVE $\forall A_i, A_i \in V \cup \Sigma$
AGGIUNGO LE REGOLE
 $A \rightarrow A_1 u_1$
 $A_1 \rightarrow A_2 u_2$
...
 $A_{k-2} \rightarrow A_{k-1} u_k$
RIMUOVO LA REGOLA $A \rightarrow u_1 u_2 \dots u_k$ INIZIALE
5. PER OGNI REGOLA $A \rightarrow u_1 u_2$ DOVE $u_1, u_2 \in V \cup \Sigma$ SCRIVO LA REGOLA $U_1 \rightarrow u_1$ SE $u_1 \in \Sigma$ DOVE $U_1 \in V$
SOSTITUISCO LA PRIMA REGOLA CON $A \rightarrow U_1 u_2$

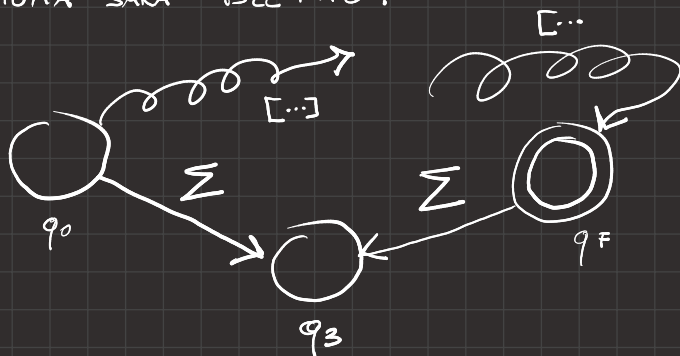
1 Automi

10 Points

- Sia $A = (Q, \Sigma, \delta, q_0, \{q_f\})$ un DFA, e si supponga che per ogni $a \in \Sigma$ si abbia $\delta(q_0, a) = \delta(q_f, a)$.
 - Si mostri che per ogni $w \neq \varepsilon$ si ha $\delta^*(q_0, w) = \delta^*(q_f, w)$, dove δ^* è la funzione di transizione estesa.
 - Si mostri che se x è una stringa non vuota in $L(A)$, allora x^k appartiene ad $L(A)$ per ogni $k > 0$.
- Definire la nozione di Push-Down Automaton (PDA). Dimostrare che un linguaggio è acontestuale se e solo se esiste un PDA che lo riconosce.

① $A = (Q, \Sigma, \sigma, q_0, \{q_f\})$
 $\sigma(q_0, a) = \sigma(q_f, a) \quad \forall a \in \Sigma$

● L'AUTOMA SARA' DEL TIPO:



LA FUNZIONE DI TRANSIZIONE ESTESA HA LE SEGUENTI PROPRIETÀ:

$$\sigma^*(q, a) = \sigma(q, a)$$

$$\sigma^*(q, aw) = \sigma^*(\sigma(q, a), w)$$

QUINDI, POENDO $w = xy \Rightarrow$

$$\begin{aligned} \sigma^*(q_0, xy) &= \\ \sigma^*(\sigma(q_0, x), y) &\text{ ma } \dots \sigma(q_0, x) = \sigma(q_f, x) \\ &= \sigma^*(\sigma(q_f, x), y) \quad \square \end{aligned}$$

● UNA STRINGA È ACCETTATA DA UN DFA SE SANO $r_1, \dots, r_{k+1}$ GLI STATI ATTRAVERSATI DURANTE LA COMPUTAZIONE

$$q_0 = r_1$$

$$\forall i \ 1 \leq i \leq k \quad r_{i+1} = \sigma(r_i, a_i)$$

$$r_{k+1} \in F$$

IN QUESTO CASO $(F = \{q_f\})$

$$x^k \in \Sigma \quad (\text{PERCHÉ L'OPERATORE POTENZA È CHIUSO IN REG})$$

$$\sigma^*(q_0, x^k) = \sigma^*(q_f, x^k) \quad \text{PER QUANTO VISTO PRIMA}$$

PROCEDIAMO PER INDUZIONE.

PER $i = 1$ $x \in L(A)$ VERO

PER $i = k - 1$ VERO PER IPOTESI

PER $i = k$ SAPIAMO CHE: $x^{k-1} \in L(A)$

$$x^k = \left[\underset{\uparrow}{x^{k-1}} \circ \underset{\uparrow}{x} \right]$$

$\in L(A) \quad \in L(A) \Rightarrow x^k \in L(A)$

② • UN PDA È UNA SESTUPLA $(Q, \Sigma, \Gamma, \sigma, q_0, F)$
DOVE Q È L'INSIEME DEGLI STATI, Σ È L'ALFABETO DELL'AUTOMA,
 Γ È L'ALFABETO DEL NASTRO, $\Sigma \cap \Gamma = \emptyset$, q_0 È LO
STATO INIZIALE, $F \subseteq Q$ È L'INSIEME DEGLI STATI FINALI.
IL NASTRO È ACCESSIBILE SOLO IN EINA E SONO
PRATICABILI OPERAZIONI DI PUSH, POP E SOSTITUZIONE.

$LE FCL \iff L \in L(PDA)$ DIMOSTRIAMO AMBO I VERSI

$L \in FCL \Rightarrow L \in L(PDA)$

$L \in FCL \Rightarrow \exists G = (V, \Sigma, R, S) \text{ CFG IN CNF, COSTRUIAMO UN PDA } D \text{ CON}$

$Q_D = q_{START}, q_{FINISH}, q_{LOOP}, \dots \text{ ALTRI STATI } \dots$

$$\sigma_D = \begin{cases} \sigma(q_{START}, \epsilon, \epsilon) = (q_{LOOP}, S\$) \\ (q_{LOOP}, w) = \sigma(q_{LOOP}, A, \epsilon) \quad \forall A \rightarrow w \in R \text{ di } G \\ (q_{FINISH}, \epsilon) = \sigma(q_{LOOP}, \$, \epsilon) \\ (q_{LOOP}, a) = \sigma(q_{LOOP}, a, a) \end{cases}$$

$$\Sigma_D = (\Sigma, \epsilon)$$

$$\Gamma_D = V \cup \Sigma \cup S \cup \{\$, \}$$

$$F_D = q_{FINISH}$$

$$q_{0D} = q_{START}$$



$L \in FCL \Leftarrow L \in L(PDA)$

$$L \in L(PDA) \Rightarrow \exists P \text{ PDA } \wedge L(P)$$

CREO UN NUOVO PDA P' CHE VUOTA LO STACK PRIMA DI

TERMINARE UNA COMPUTAZIONE E NON ESEGUE MAI SOSTITUZIONI.

P E P' E' FACILE INTUIRE RICONOSCONO GLI STESSI LINGUAGGI.

CREO UNA GRAMMATICA G DOVE

$$V_G = A_{p,q} \text{ DOVE } p, q \text{ SONO GLI STATI DI } P' (P'[q])$$

• AFFERMAZIONE. $A_{p,q}$ DERIVA TUTTE LE STRINGHE x | x PORTA L'AUTOMA DALLO STATO p ALLO STATO q CON STACK VUOTO.

* DIMOSTRAZIONE OMESSA *

$A_{q_{\text{START}}, q_{\text{FINISH}}}$ DERIVA TUTTE LE STRINGHE x CHE PORTANO L'AUTOMA DALLO q_{START} A q_{FINISH} SVUOTANDO LO STACK.




FreddyLix17 on Feb 4, 2025 Collaborator

- Si consideri la seguente grammatica G :

$$\begin{aligned}
 S &\rightarrow aB \mid bA \mid \epsilon \\
 A &\rightarrow aS \mid bAA \\
 B &\rightarrow aBB \mid bS
 \end{aligned}$$

Si fornisca una derivazione sinistra della stringa $bbaaab$. Si determini un PDA che riconosce lo stesso linguaggio generato da G .

- Definire le grammatiche acontestuali e dimostrare che ogni grammatica acontestuale ne ammette una equivalente in forma normale di Chomsky.

Mostrare che in una grammatica in forma normale di Chomsky ogni derivazione di una stringa w tale che $|w| = n$ richiede al più $2n - 1$ passi (per ogni $n \geq 1$).

↑ 2
😊

1b)

• CREO UN PDA P tale che:

$Q_P = q_{\text{LOOP}}, q_{\text{START}}, q_{\text{FINISH}}, \dots$ ALTRI STATI UTILI ALLA COMPUTAZIONE...

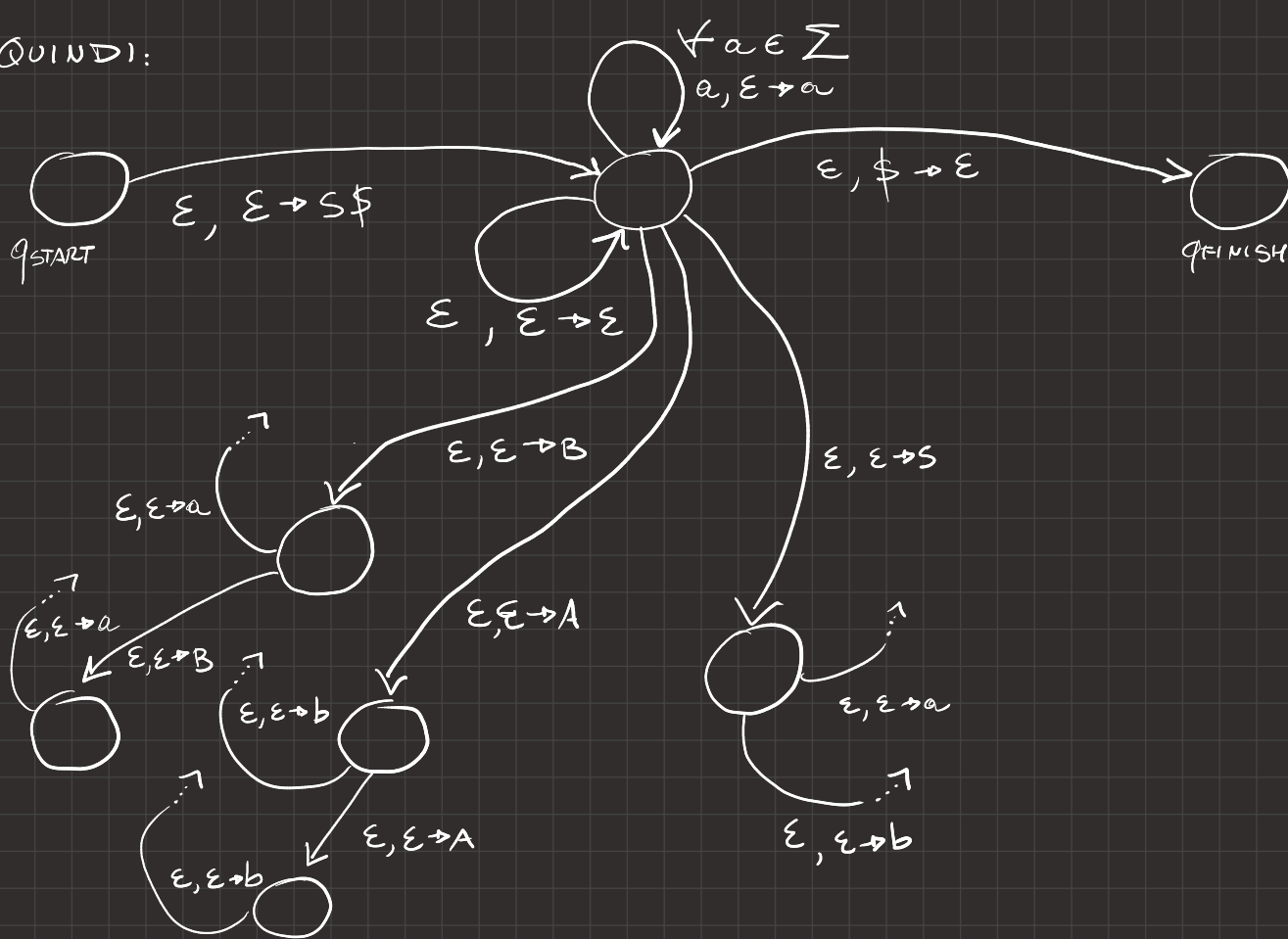
$$\sigma_P = \begin{cases} \sigma_P(q_{\text{START}}, \epsilon, \epsilon) = (q_{\text{LOOP}}, S\$) \\ \sigma_P(q_{\text{LOOP}}, A, \epsilon) = (q_{\text{LOOP}}, w, \epsilon) \quad \forall A \rightarrow w \in R \\ \sigma_P(q_{\text{LOOP}}, a, a) = (q_{\text{LOOP}}, a) \\ \sigma_P(q_{\text{LOOP}}, \$, a) = (q_{\text{FINISH}}, \epsilon) \end{cases}$$

$$\Sigma_P = \Sigma$$

$$\Gamma_P = \Sigma \cup V \cup S \cup \{\$, \}$$

$\{q_{\text{START}}\}, \{q_{\text{FINISH}}\}$ SONO STATO INIZIALE E FINALE

QUINDI:



INDICE CHE SI TORNA
A 9 LOOP

1a)

$$S \rightarrow bA \rightarrow bbAA \rightarrow bbaSA \rightarrow bbaqaB \rightarrow (bbaqa)$$

2a)

$$G \text{ CFG} = (V, \Sigma, R, S)$$

DOVE V : INSIEME VARIABILI

Σ : INSIEME TERMINATORI

R : REGOLE GRAMMATICHE

S : STATO INIZIALE

$$V \cap \Sigma = \emptyset$$
$$S \in V$$

- PER OGNI G POSSO TROVARE G' GRAMMATICA

ACONTESTUALE IN FORMA NORMALE DI PROMPT, SEGUE ALGORITHM
PARTENDO DA G

1. CREO UN NUOVO STATO S_0 , CREO LA REGOLA $S \rightarrow S_0$

2. ELIMINO LE REGOLE TIPO $A \rightarrow \epsilon$

PER OGNI ALTRA REGOLA TIPO $B \rightarrow \epsilon A \epsilon \epsilon$ AGGIUNGO
COMBINAZIONI TIPO $B \rightarrow \epsilon A \epsilon A \epsilon \dots$

3. ELIMINO REGOLE TIPO $A \rightarrow B$ DOVE $A, B \in \Sigma \cup V$

PER OGNI REGOLA $B \rightarrow a$ DOVE $a \in \Sigma$

CREO LA REGOLA $A \rightarrow a$, ELIMINO $B \rightarrow A$

4. PER TUTTE LE REGOLE TIPO $A \rightarrow u_1 \dots u_k$ DOVE

$u_1, \dots, u_k \in \Sigma \cup V$, $A \in V$

CREO $A_1, \dots, A_{k-1}$

CREO LE REGOLE

$A \rightarrow A_1 u_1$

...

$A_{k-2} \rightarrow A_{k-1} u_k$

ELIMINO $A \rightarrow u_1 \dots u_k$

5. PER TUTTE LE REGOLE TIPO $A \rightarrow u_1 u_2$ DOVE

$u_1, u_2 \in V \cup \Sigma$, $A \in V$

CREO $U_1 \rightarrow u_1$ SE $u_1 \in \Sigma$ | $U_1 \in V$

CREO $A \rightarrow U_1 u_2$, ELIMINO L'ORIGINALE. $\square$

2b) PROViamo PER INDUZIONE.

a) PER $i=1$ *VERO*

b) SUPPONIAMO VERO FINO A $m-1$

c) PROViamo PER m

DATO CHE G È IN CNF, $S \rightarrow AB$ È STATO IL PRIMO
PASSO, DOVE $V = AB$, PER IPOTESI INDUTTIVA

A RICHIEDE $2(m-k)-1$ PASSI E B RICHIEDE $2(m-(m-k))-1$
PASSI QUINDI AB RICHIEDE $2m-1$ PASSI.

1 Automi

- Si classifichino i seguenti linguaggi come i) regolari, oppure ii) acontestuali ma non regolari, oppure iii) non acontestuali. In ciascun caso, si giustifichi la risposta.

(a) $\{w\#u : w, u \in \{0,1\}^* \text{ e } |w| = |u|\}$.

(b) $\left\{ w \in \{0,1\}^* : \begin{array}{l} w \text{ contiene un numero dispari di simboli '0'} \\ \text{ed un singolo simbolo '1'} \end{array} \right\}$.

(c) $\left\{ w \in \{0,1,2\}^* : \begin{array}{l} w \text{ contiene più simboli '0' che simboli '1'} \\ \text{e più simboli '0' che simboli '2'} \end{array} \right\}$.

- Si dimostri che un linguaggio è regolare se e solo se esiste un NFA che lo riconosce.

a) $L_1 = \{w\#u : w, u \in \{0,1\}^* \text{ e } |w| = |u|\}$

VERIFICHIAMO LA REGOLARITÀ, LA STRINGA GENERICA DI L_1 È $w = xyz$

$|y| > 0$ PER IL PUMPING LEMMA QUINDI

y PUO' ESSERE IL CARATTERE $\# \Rightarrow xy^iz \notin L_1$

IL CARATTERE $\#$ È PARTE DI $x \Rightarrow xy^iz \notin L_1$

IL CARATTERE $\#$ È PARTE DI $z \Rightarrow xy^iz \notin L_1$

NON CONTENERE $\#$ E CONTENERE ZERI $\Rightarrow xy^iz \notin L_1$

L_1 NON È REGOLARE

b) $L_2 = \left\{ w \in \{0,1\}^* : \begin{array}{l} w \text{ CONTIENE UN NUMERO DISPARI DI} \\ \text{SIMBOLI '0' ED UN SINGOLO '1'} \end{array} \right\}$

CONSIDERIAMO I DUE LINGUAGGI L E L'

$L = \left\{ w \in \{0,1\}^* : \begin{array}{l} w \text{ CONTIENE UN NUMERO DISPARI} \\ \text{DI ZERO} \end{array} \right\}$

$L' = \left\{ w \in \{0,1\}^* : \begin{array}{l} w \text{ CONTIENE UN SOLO CARATTERE} \\ \text{'1'} \end{array} \right\}$

$L_2 = L \cap L'$, $L \in \text{REG}$, $L' \in \text{REG} \Rightarrow L_2 \in \text{REG}$

c) $L_3 = \left\{ w \in \{0,1,2\}^* : \begin{array}{l} w \text{ CONTIENE PIÙ SIMBOLI '0' CHE} \\ \text{SIMBOLI '1' E PIÙ SIMBOLI '0' CHE} \\ \text{SIMBOLI '2'} \end{array} \right\}$

APPLICHIAMO IL PUMPING LEMMA.

$w = xyz$

$|x| \leq p$

$|xy| \leq p$

$|y| > 0$

NO

1 Automi

- Si classifichino i seguenti linguaggi come i) regolari, oppure ii) acontestuali ma non regolari, oppure iii) non acontestuali. In ciascun caso, si giustifichi la risposta.

(a) $\{w\#u : w, u \in \{0, 1\}^* \text{ e } |w| = |u|\}$.

(b) $\left\{ w \in \{0, 1\}^* : \begin{array}{l} w \text{ contiene un numero dispari di simboli '0'} \\ \text{ed un singolo simbolo '1'} \end{array} \right\}$.

(c) $\left\{ w \in \{0, 1, 2\}^* : \begin{array}{l} w \text{ contiene più simboli '0' che simboli '1'} \\ \text{e più simboli '0' che simboli '2'} \end{array} \right\}$.

- Si dimostri che un linguaggio è regolare se e solo se esiste un NFA che lo riconosce.

1

a) $L_1 = \{w\#u : w, u \in \{0, 1\}^* \text{ e } |w| = |u|\}$

Linguaggi di questo tipo generalmente non sono REGOLARI perché un DFA riconosce linguaggi che controllano PRESENZA/PARITÀ/CONTEGGI < ∞ ma non questo. Quindi mi aspetto non sia regolare.

APPLICO IL PUMPING LEMMA.

w = STRINGA GENERICA IN $L_1 = xyz$

dato che $|w| = |u|$ le stringhe sono tipo: $0^p \# 0^p$ ma

$|xy| \leq p$ e $|y| > 0$ allora una stringa generica sarà:

$0^{p-k} 0^k \# 0^p$ quindi $0^{p-k} (0^k)^0 \# 0^p$ dovrebbe appartenere

ad L_1 ma così non è quindi $L_1 \notin \text{REG}$

Verifichiamo ora se appartiene agli ACONTESTUALI, mi aspetto di sì perché i confronti sono fattibili con il PDA.

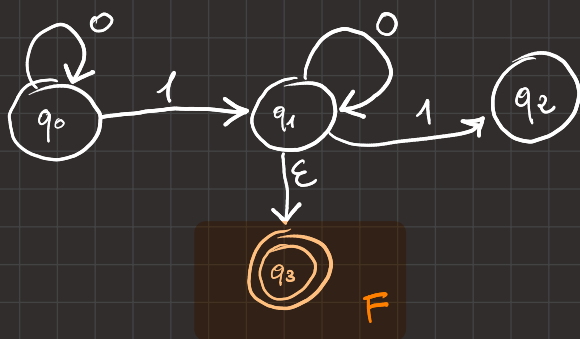
Infatti posso creare un automa PDA P che legge x , una generica stringa di L_1 , PUSHA un carattere 'x' nel nostro quando legge un carattere di w , POPPA un carattere dallo stack quando legge un carattere di u dopo aver letto '#'.
e fine computazione la pila è vuota ok.

Quindi $(L_1 \in \text{CFL}) \wedge (L_1 \notin \text{REG})$

b) $\{w \in \{0, 1\}^* : w \text{ CONTIENE '0' IN NUMERO DISPARI} \wedge \text{E UN CARATTERE '1'}\}$

Mi aspetto che questo linguaggio sia REGOLARE perché i DFA riescono a verificare presenza e parità.

CREO UN AUTOMA DFA...



Quindi $L_2 \in REG$

e) $L_3 = \{ w \in \{0,1,2\}^* : w \text{ contiene più simboli '0' che simboli '1' e più simboli '1' che simboli '2' } \}$

Mi aspetto che questo linguaggio sia non REGOLARE e non ACONTESTUALE perché né DFA né PDA sono in grado di operare confronti multipli... Dimostrerò usando il PUMPING LEMMA per i linguaggi ACONTESTUALI.

w è una stringa di L_3 , $w = 0^{p+1}1^p2^p$

$w = xyzk$

$|yzk| \leq p$ lunghezza del pumping, yzk ricade o negli 0, o negli 1, o nei 2 o è coinvolto tra questi.

se ricade negli 0

per $i=0$ $\#0 < \#1$

se ricade negli 1

$\#1 > \#0$

se ricade nei 2

$\#2 > \#0$

se ricade e coinvolto $\#2 > \#0$ o $\#1 > \#0$

quindi $L_3 \notin CFL$

$$[L \in \text{REG} \Leftrightarrow \exists \text{NFA } N, L(N)]$$

IDEA DI DIMOSTRAZIONE:

$$(\Rightarrow) L \in \text{REG} \Rightarrow L \in \mathcal{L}(\text{DFA}).$$

$$\mathcal{L}(\text{DFA}) = \mathcal{L}(\text{NFA}) \Rightarrow L \in \text{REG}, L \in \mathcal{L}(\text{NFA})$$

$$L \in \mathcal{L}(\text{NFA}) \Rightarrow \exists \text{NFA } N, L(N)$$

$$(\Leftarrow) \exists \text{NFA } N, L(N) \Rightarrow L \in \mathcal{L}(\text{NFA})$$

$$L \in \mathcal{L}(\text{NFA}) \Rightarrow L \in \mathcal{L}(\text{DFA})$$

$$L \in \mathcal{L}(\text{DFA}) \Rightarrow L \in \text{REG}.$$

TUTTO VERO PERCHÉ $\mathcal{L}(\text{NFA}) = \mathcal{L}(\text{DFA})$ * DIMOSTRAZIONE OMESSA *

1 Parte Prima

10 Points

☒ Dimostrare che il seguente linguaggio su alfabeto $\{\#, 1\}$ non è regolare:

$$\{1^i \# 1^j \# 1^{i+j} : i, j \geq 1\}.$$

☒ Dimostrare che se un linguaggio è regolare allora esiste una espressione regolare che lo descrive.

Mi aspetto che questo linguaggio non sia regolare perché i DFA non sono in grado di ottenere confronti.

APPLICO IL PUMPING LEMMA

Sia w una stringa di L , $w = 1^P \# 1^P \# 1^{2P}$, $w = xyz$

$$|xy| \leq P$$

$$|y| > 0$$

quindi xy devono ricoprire per forza "nel primo 1^P "

$$w = xyz \Rightarrow w = \overset{x}{1^{k-1}} \overset{y}{1^1} \overset{z}{1^{P-k}} \# 1^P \# 1^{2P} \text{ per } k > 0 \wedge k \leq P$$

la stringa $w = 1^{k-1} (1^1)^i 1^{P-k} \# 1^P \# 1^{2P} \notin L$ per $i = 0$

Quindi $L \notin \text{REG}$

1 Automi

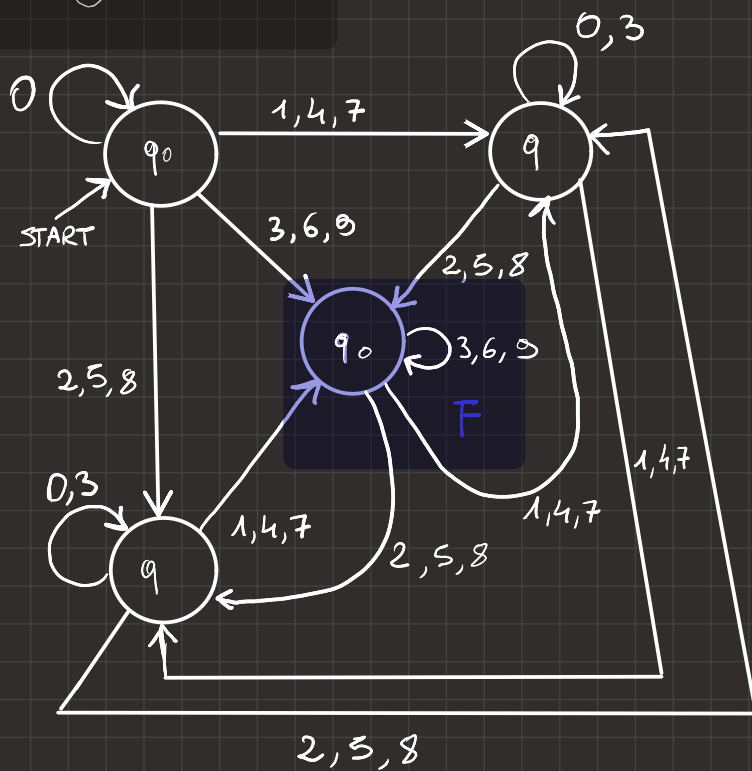
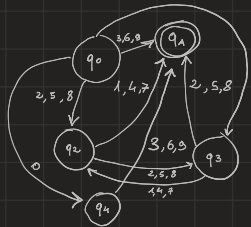
10 Points

- Progettare un DFA che riconosca il linguaggio di stringhe su alfabeto $\Sigma = \{0, 1, \dots, 9\}$ tali che la somma delle cifre sia un multiplo di 3. Provare la correttezza del DFA proposto.
- Enunciare e dimostrare il *pumping lemma* per linguaggi regolari. Fornire un esempio di applicazione.



$$\Sigma = \{0, 1, \dots, 9\}$$

BRUTTA



DIMOSTRAZIONE:

DIMOSTRIAMO PER INDUZIONE SU n LUNGHEZZA INPUT

CASO BASE.

CON INPUT DI LUNGHEZZA 1 L'AUTOMA FUNZIONA

IPOTESI INDUTTIVA.

FUNZIONA PER INPUT DI LUNGHEZZA n

PASSO INDUTTIVO.

SE L'AUTOMA FUNZIONA SU STRINGHE LUNGHE n

DOPO AVER LETTO n CARATTERI

0 SONO IN q_0 (SE TUTTI I CARATTERI ERANO "0")
 0 SONO IN q_1 (SE DEVE LEGGERE $\{2, 5, 8\}$ PER OTTENERE UN MULTIPLO DI 3)
 0 SONO IN q_2 (SE DEVE LEGGERE $\{1, 4, 7\}$)
 0 SONO IN q_4 (SE HO UN MULTIPLO DI 3)
 LEGGENDO L' $m+1$ ESIMO CARATTERE

NO

DIMOSTRAZIONE:

LA FUNZIONE DI TRANSIZIONE σ SARA' :

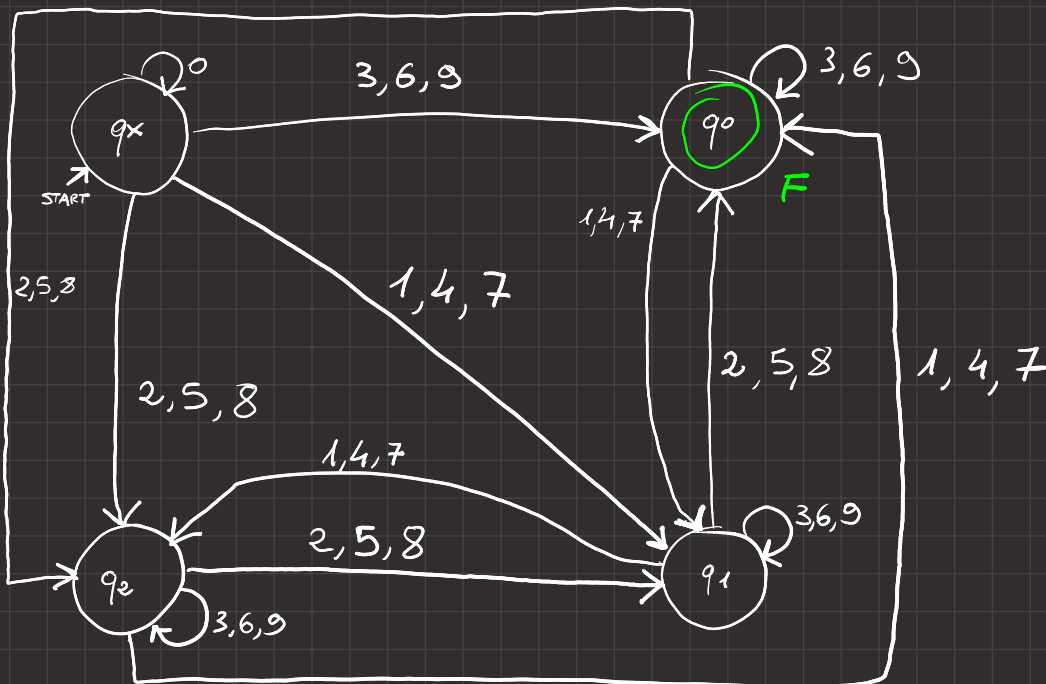
$$\sigma(q_i, a) = q_j \iff a + i = \text{mod}(j)$$

NO

1 Automi

10 Points

- Progettare un DFA che riconosca il linguaggio di stringhe su alfabeto $\Sigma = \{0, 1, \dots, 9\}$ tali che la somma delle cifre sia un multiplo di 3. Provare la correttezza del DFA proposto.
- Enunciare e dimostrare il *pumping lemma* per linguaggi regolari. Fornire un esempio di applicazione.



DIMOSTRAZIONE

LA FUNZIONE DI TRANSIZIONE σ SARA' DEL TIPO:

$$\sigma(q_i, a) = q_j \iff i + a = \text{mod}(3)$$

$$\sigma(q_x, a) = q_1 \text{ se } a = 1, 4, 7$$

$$\delta(q_x, a) = q_2 \text{ se } a = 2, 5, 8$$

$$\delta(q_x, a) = q_3 \text{ se } a = 3, 6, 9$$

SAPENDO QUESTO DIMOSTRIAMO PER INDUZIONE SU k (LUNGHEZZA DELL'INPUT)

• CASO BASE

SE $k = 1$ L'AUTOMA FUNZIONA, ACCETTA SOLO I MULTIPLI DI 3

• IPOTESI INDUTTIVA

SUPPONIAMOLO FUNZIONANTE FINO AD INPUT LUNGHI $k = n$

• PASSO INDUTTIVO

SE HO UN INPUT DI LUNGHEZZA $n+1$ E HO SUPPOSTO L'AUTOMA FUNZIONANTE SU INPUT DI LUNGHEZZA n , DOPO n PASSI MI TROVERO' O SU q_1 O SU q_2 O SU q_0

- SARO' SU q_1 SE LA SOMMA E' UN NUMERO $x = 1 \bmod(3)$
 // q_2 // $x = 2 \bmod(3)$
 // q_0 // $x = 0 \bmod(3)$

IL PROSSIMO VALORE IN INPUT ($n+1$ ESIMO VALORE) E' UN NUMERO DA 1 A 9

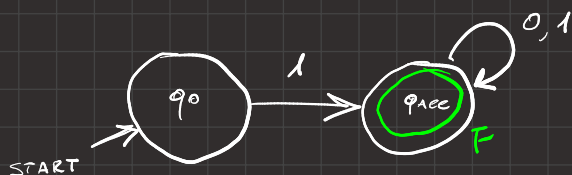
SUPPONIAMO DI ESSERE IN q_i E AVERE INPUT a
 ANDREMO IN $q_j \iff i+a = j \bmod(3)$

1 Parte Prima

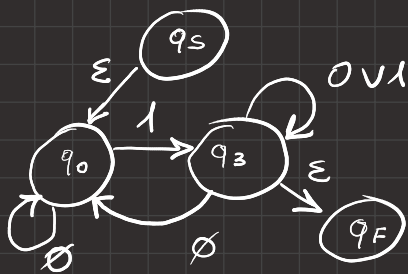
10 Points

- Si consideri l'espressione regolare $1\Sigma^* = 1(0\cup 1)^*$. Convertire tale espressione regolare in una grammatica acontestuale. Dimostrare la correttezza.
- Enunciare e dimostrare il pumping lemma per linguaggi regolari. Fornire un esempio di utilizzo.

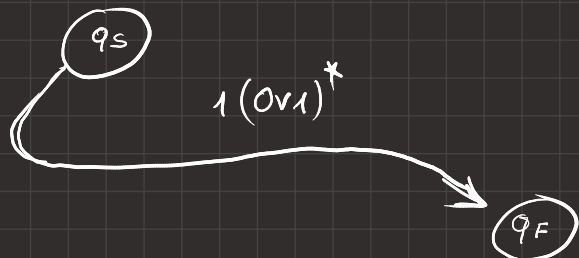
SAPPIAMO CHE $L(\text{DFA}) \subseteq \text{CFL}$ QUINDI POSSO CREARE UN AUTOMA, DIMOSTRARE LA SUA CORRETTEZZA E POI DA QUESTO DERIVARE LA CFG



DIMOSTRO CHE QUESTO AUTOMA E' VALIDO, POSSO INFATTI TRASFORMARLO IN UN GNFA



CHE POSSIAMO RIDURRE A:



QUINDI E' DIMOSTRATO CHE L'AUTOMA DISEGNATO IN PRINCIPIO RICONOSCE L'ESPRESSIONE $1(01)^*$

DALL'AUTOMA COSTRUIAMO UNA GRAMMATICA CFG G

$$\mathcal{L}(q_i) = V_i, \quad \mathcal{L}(q_{\text{START}}) = S, \quad \mathcal{L}(q_{\text{FINISH}}) = f$$

$$\sigma(q_i, a) = q_j \iff V_i \rightarrow a V_j$$

LA GRAMMATICA SARA' QUINDI

$$S \rightarrow 1 f$$

$$f \rightarrow 1 f \mid 0 f \mid \epsilon$$

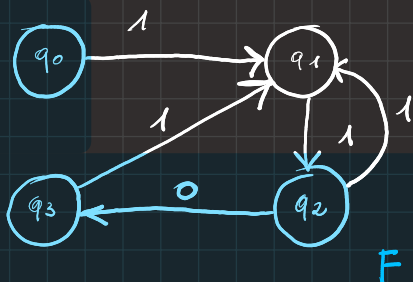
1 Automi

10 Points

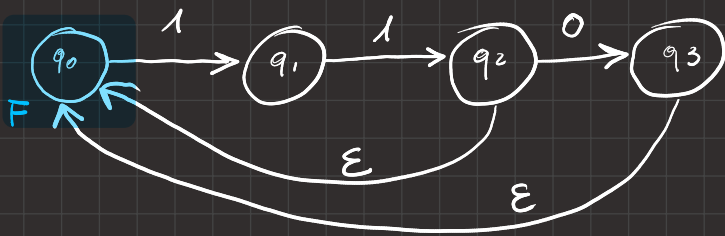
- Sia $L = \{11, 110\}^*$. Costruire un NFA N con 4 stati che riconosca L . Convertire l'NFA N in un DFA M equivalente.
- Enunciare e dimostrare il *pumping lemma* per linguaggi regolari. Fornire un esempio di utilizzo.

$$L = \{11, 110\}^*$$

DISEGNAMO PRIMA IL DFA.



QUESTO E' GIA' DI SUO UN NFA POICHE' UN DFA E' UN NFA "SPECIALIZZATO" CON SOLE TRANSIZIONI DETERMINISTE



ALTRIMENTI QUESTO È UN NFA CORRISPONDENTE AL DFA SOPRA.

1 Automi

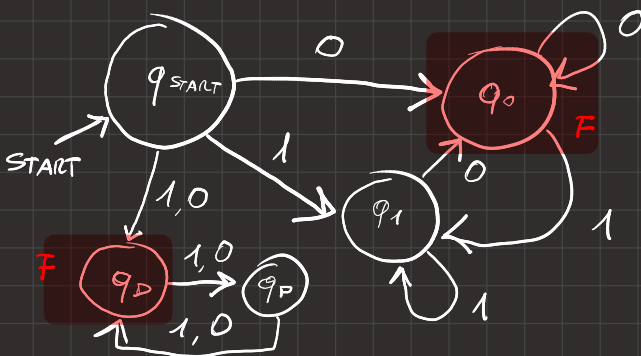
10 Points

- Dimostrare che il linguaggio $L = \{w : w \text{ termina con } 0 \text{ oppure ha lunghezza pari}\}$ è regolare.
- Definire le grammatiche acontestuali e dimostrare che ogni grammatica acontestuale ne ammette una equivalente in forma normale di Chomsky. Mostrare che in una grammatica in forma normale di Chomsky ogni derivazione di una stringa w tale che $|w| = n$ richiede al più $2n - 1$ passi (per ogni $n \geq 1$).

SAPPIAMO CHE $Reg = L(DFA)$ QUINDI UN LINGUAGGIO L PER ESSERE REGOLARE DEVE ESISTERE UN DFA D t.c. $L(D)$

UN DFA È IN GRADO DI VERIFICARE PRESENZE E PARITÀ QUINDI PRESUMO CHE $L \in REG$.

VERIFICHIAMO.



QUESTO CHE HO DISEGNATO È UN NFA CHE RICONOSCE L , DATO CHE $L(DFA) = L(NFA) = REG$ E $L(NFA) \Rightarrow L \in REG$

2^a parte... IN UNA CFG IN CNF OGNI DERIVAZIONE DI UNA STRINGA w t.c. $|w| = n$ RICHIEDE ALPIÙ $2n - 1$ PASSI

DIMOSTRO PER INDUZIONE SU n (LUNGHEZZA STRINGA)

• CASO BASE.

PER $n = 1$ DATO CHE SIAMO IN CNF AVRO' USATO PER FORZA LA REGOLA $S \rightarrow \epsilon$ QUINDI UN SOLO PASSO

VERO

• IPOTESI INDUTTIVA

PER $n = k$ SUPPONGO VERO

• PASSO INDUTTIVO

PROVO PER $k+1$

DATO CHE SONO IN CNF, LA PRIMA REGOLA USATA SARÀ STATA

$S \rightarrow AB$ SE DERIVO A COMPIO $2(m+1-k)-1$ PASSI, SE DERIVO B COMPIO $2k-1$ PASSI

PERCHÉ $m-k \leq m$ E $k < m$

QUINDI PER DERIVARE COMPIO $2(m+1-k)-1 + 2k-1 =$

$$2m+2-2k-1+2k-1 = 2m = 2m \text{ PASSI}$$

DOBBIAMO POI AGGIUNGERE IL PRIMO PASSO, QUINDI:

$$2m+1 = 2(m+1)-1 = 2m+2-1$$

□

1 Automi

- Utilizzare il *pumping lemma* per mostrare che il seguente linguaggio su alfabeto $\Sigma = \{a, b, c\}$ non è regolare:

$$L = \{c^4 a^n b^m : n \geq m\}.$$

- Dimostrare che un linguaggio è regolare se e solo se esiste una espressione regolare che lo descrive.

• "FARE CONFRONTI" NON È COSA DA DFA, QUINDI SUPPONGO CHE QUESTO LINGUAGGIO NON SIA IN REG.

DIMOSTRIAMO CON IL PUMPING LEMMA.

PRENDO UNA STRINGA $w = c^4 a^p b^p$ DOVE p È LA LUNGHEZZA DEL PUMPING

Y PUÒ RILEADERE IN DIVERSI PUNTI

SULLE "c" O A CAVALLO DELLE "c" $\Rightarrow w \notin L \Rightarrow L \not\subseteq \text{REG}$

SULLE "b" E SE $i=0 \Rightarrow w \notin L \Rightarrow L \not\subseteq \text{REG}$

$\Rightarrow L \not\subseteq \text{REG}$

• IDEA DI RAGIONAMENTO.

$$\text{REG} = \mathcal{L}(\text{DFA})$$

$$\mathcal{L}(\text{DFA}) = \mathcal{L}(\text{GNFA})$$

$$\mathcal{L}(\text{GNFA}) = \mathcal{L}(\text{RE})$$

$$\Rightarrow \text{REG} = \mathcal{L}(\text{RE})$$



- Enunciare e dimostrare il *pumping lemma* per linguaggi regolari. Fornire un esempio di utilizzo.

$$R = \begin{aligned} S &\rightarrow ZZUDTT \mid ZT \mid \varepsilon \\ Z &\rightarrow ZZ \mid 0 \\ U &\rightarrow UUZ \mid 1 \\ D &\rightarrow DD \mid 2 \\ T &\rightarrow TT \mid 3 \end{aligned} \quad G = (V = \{S, T, U, D, Z\}, \Sigma = \{0, 1, 2, 3, \varepsilon\}, S, R = \dots)$$

1 Automi

10 Points

- Mostrare che la seguente grammatica è ambigua:

$$S \rightarrow TbT$$

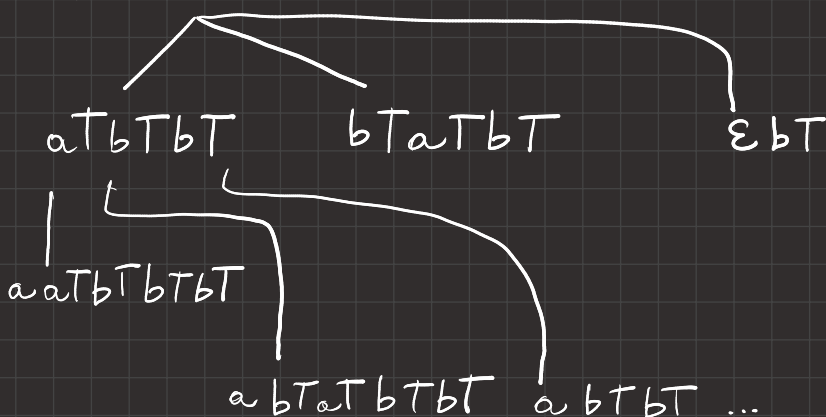
$$T \rightarrow aTbT | bTaT | \epsilon.$$

- Dimostrare che se un linguaggio è regolare allora esiste una espressione regolare che lo descrive.

UNA GRAMMATICA CONTESTUALE SI DICE AMBIGUA SE ESISTONO 2 O PIU' DERIVAZIONI SINISTRE. VERIFICHIAMO

$$S = TbT$$

$$TbT =$$



NON CONTINUO LA GRAMMATICA È AMBIGUA AVERO' INFATTI UNA DERIVAZIONE SINISTRA TIPO aa... E UNA ab... .

No

1 Automi

10 Points

- Mostrare che la seguente grammatica è ambigua:

$$S \rightarrow TbT$$

$$T \rightarrow aTbT | bTaT | \epsilon.$$

- Dimostrare che se un linguaggio è regolare allora esiste una espressione regolare che lo descrive.

1) PER VERIFICARE CHE UNA GRAMMATICA È AMBIGUA DOBBIAMO AVERE DUE DERIVAZIONI SINISTRE PER LA STESSA STRINGA.

$$S \rightarrow TbT \rightarrow bT \rightarrow baTbT \rightarrow bab$$

$$S \rightarrow TbT \rightarrow bTaTb \rightarrow bab$$

LA GRAMMATICA È AMBIGUA

2) IDEA DI RAGIONAMENTO.

$$\left. \begin{array}{l} \text{REG} = \mathcal{L}(\text{DFA}) \\ \mathcal{L}(\text{DFA}) = \mathcal{L}(\text{GNFA}) \\ \mathcal{L}(\text{GNFA}) = \mathcal{L}(\text{RE}) \end{array} \right\} \Rightarrow \text{REG} = \mathcal{L}(\text{RE})$$

1 Automi

10 Points

- Sia C un linguaggio *context-free* ed R un linguaggio regolare. Mostrare che il linguaggio $C \cap R$ è *context-free*. Usare questo risultato per mostrare che il linguaggio L delle stringhe $w \in \{a, b, c\}^*$ tali che w contiene lo stesso numero di a , b e c non è *context-free*.
- Dimostrare che un linguaggio è *acontestuale* se e solo se esiste un PDA che lo riconosce.

SAPPIAMO CHE $L(DFA) \subset CFL$ EIOE
 $REG \subset CFL$

QUINDI NON E' DETTO CHE UN LINGUAGGIO ACONTESTUALE SIA ANCHE REGOLARE

PRENDENDO DUE LINGUAGGI C CONTEXT-FREE E
 R REGOLARE

$C \cap R$ E' ACONTESTUALE **NO**

1 Automi

- Stabilire se i seguenti linguaggi sono regolari:

- $\{a^n b^m c^n d^m : n, m \geq 0\}$.
- $\{a^n c^m b^n : n > m \geq 0\}$.
- $\{w \in \{a, b, c\}^* : \#_a(w) = \#_b(w) \text{ e } \#_a(w) > \#_c(w)\}$, dove $\#_x(w)$ indica il numero di occorrenze del simbolo $x \in \{a, b, c\}$ nella stringa w .
- $\{w \# t \# w^R : w, t \in \{a, b\}^* \text{ e } |w| = |t|\}$, dove w^R rappresenta la stringa w rovesciata.

- Dimostrare che un linguaggio è *acontestuale* se e solo se esiste un PDA che lo riconosce.

a) $L = \{a^n b^m c^n d^m : n, m \geq 0\}$

Mi aspetto che il linguaggio L non sia REGOLARE dato che i DFA non possono eseguire confronti.

prendo $w = a^p b^p c^p d^p$ dove p = lunghezza pumping

Sappiamo che $|xy| \leq p$ e che $|y| > 0$

y dove ricorrendo nel passo con tutte "a" se $i=0$ allora

$w \notin L \Rightarrow L \notin REG$

b) $L_2 = \{a^n c^m b^n : n > m \geq 0\}$

per lo stesso motivo del punto a) mi aspetto che $L_2 \notin REG$

prendo $w = a^p c^p b^p$

y dove ricorrendo nel passo con tutte "a" se $i=0$ allora

$w \notin L \Rightarrow L \notin REG$

c) L_3

prendo $w = a^p b^p c^{p-1}$

y deve ricadere nel passo con tutte "a" e $i=0$ allora

$w \notin L \Rightarrow L \notin REG$

d) L_4

prendo $w = a^p \# t^p \# a^p$

y deve ricadere nel passo con tutte "a" e $i=0$ allora

$w \notin L \Rightarrow L \notin REG$

1 Automi

10 Points

- Considerare la grammatica G così definita:

$$S \rightarrow WbT$$

$$T \rightarrow aWbT | bVaT | \epsilon$$

$$W \rightarrow aWbW | \epsilon$$

$$V \rightarrow bVaV | \epsilon$$

Mostrare che $aabbbbbaab$ appartiene ad $L(G)$. Descrivere un PDA equivalente alla grammatica G .

- Enunciare e dimostrare il *pumping lemma* per linguaggi regolari. Fornire un esempio di utilizzo.

$$1) S \Rightarrow WbT \Rightarrow aWbWbT \Rightarrow aaWbWbWbT \Rightarrow aabbbbVaT \Rightarrow aabbbbbaaWbT \Rightarrow aabbbbbaab$$

$$2) P = PDA(Q, \Sigma_1, \Gamma, q_0, \sigma, q_{FINISH}) \text{ DA } G \text{ CFG:}$$

$$Q = \{q_0, q_{FINISH}, q_{LOOP}, [ALTRI NODI]\}$$

$$\sigma = \begin{cases} \sigma(q_0, \epsilon, \epsilon) = (q_{LOOP}, S\$) \\ \sigma(q_{LOOP}, \epsilon, A) = (q_{LOOP}, u) \Leftrightarrow A \rightarrow u \in R \\ \sigma(q_{LOOP}, \epsilon,) = (q_{FINISH}, \epsilon) \\ \sigma(q_{LOOP}, a, a) = (q_{LOOP}, \epsilon) \end{cases}$$

$$\Sigma_1 = \Sigma$$

$$\Gamma = \Sigma \cup V$$

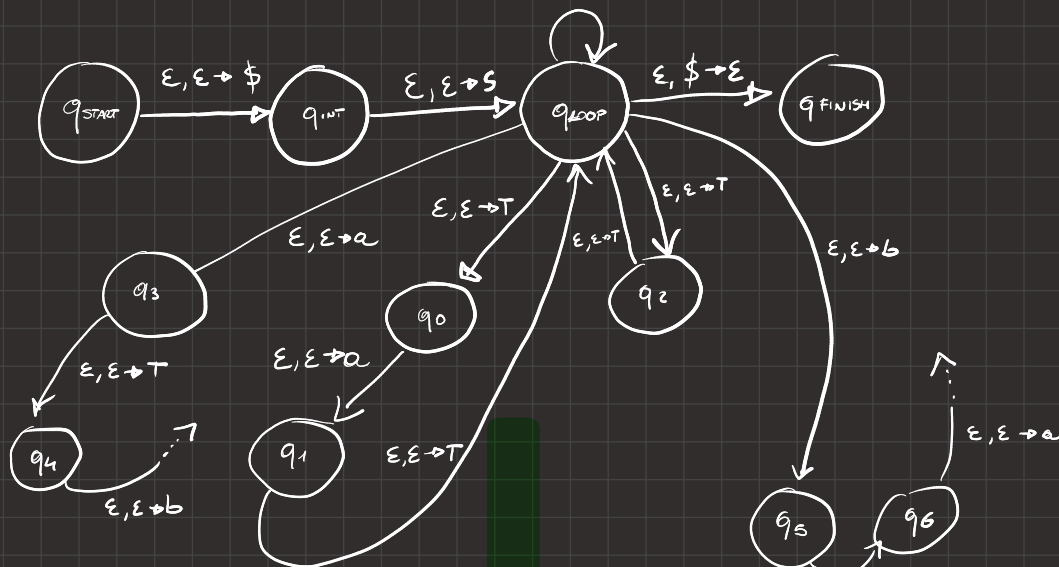
\$

NEL NOSTRO CASO

$$P = PDA(Q, \Sigma_1, \Gamma, q_0, \sigma, q_{FINISH})$$

M:

$\epsilon, T \rightarrow \epsilon$
 $\epsilon, \epsilon \rightarrow \epsilon$
 $a, a \rightarrow \epsilon$
 $b, b \rightarrow \epsilon$



STATO	INPUT	NASTRO
q _{START}	ababbbaa	ϵ
q _{LOOP}	ababbbaa	S \$
q _{LOOP}	ababbbaa	T a T \$
q _{LOOP}	ababbbaa	a T b a T \$
q _{LOOP}	bbbaa	b T a \$
q _{LOOP}	baa	b T a a \$
q _{LOOP}	ϵ	\$
q _{FINISH}	ϵ	ϵ

USANDO •

TEMPO

$S \Rightarrow T a T \Rightarrow a T b a T \Rightarrow a T b a T T \Rightarrow a T b a b T a T \Rightarrow$
 $a T b a b b T a a \Rightarrow a b a b b a a$

2 Calcolabilità

10 Points

✓ Per $k \in \mathbb{N}$, siano $L_1, \dots, L_k$ linguaggi arbitrari su un alfabeto Σ . Si assuma che: (i) $L_i \cap L_j = \emptyset$, per ogni $i \neq j$; (ii) $L_1 \cup \dots \cup L_k = \Sigma^*$; (iii) L_i è Turing-riconoscibile, per ogni $i \in [k]$. Dimostrare che i linguaggi $L_1, \dots, L_k$ sono decidibili.

✓ Dimostrare che il linguaggio $E_{TM} = \{\langle M \rangle : M \text{ è una TM ed } L(M) = \emptyset\}$ è indecidibile.

• SAPPIAMO CHE SE HO $L \in REC \wedge L \in COREC \Rightarrow L \in DEC$
 PERCHÉ POSSIAMO CREARE UNA MACCHINA DI TURING T CHE
 PRENDE IN INPUT UNA STRINGA "w" E SIMULA T', T'', LE
 MACCHINE DI TURING CHE RICONOSCONO RISPETTIVAMENTE L E
 Σ ALTERNANDOLE SE UNA DELLE DUE MACCHINE
 ACCETTA T ACCETTA',
 T ACCETTA SEMPRE PERCHÉ w SARA' IN UNO DEI
 DUE LINGUAGGI.

DATO CHE $L_i \cap L_j = \emptyset \quad \forall i \neq j$ E $L_1 \cup \dots \cup L_k = \Sigma^*$,
 I LINGUAGGI $L_1, \dots, L_k$ FORMANO UNA PARTIZIONE QUIN DI

$$L_i = \bigcup_{j=1}^{i-1} L_j \cup \left[\bigcup_{z=i+1}^k L_z \right]$$

E QUESTO È RICONOSCIBILE DATO CHE È L'UNIONE DI LINGUAGGI RICONOSCIIBILI (POTREI CREARE UNA TM M WRAPPER CHE PROVA LA STRINGA "w" PASSATA IN INPUT IN PARALLELO SU TUTTE LE MACCHINE $M_1, \dots, M_{i-1}, M_{i+1}, \dots, M_k$, MACCHINE DI TURING CHE RICONOSCONO RISPETTIVAMENTE $L_1, \dots, L_{i-1}, L_{i+1}, \dots, L_k$ SE UNA DI QUESTE ACCETTA M ACCETTA.)

L_i È TURING RICONOSCIBILE E CO-TURING RICONOSCIBILE
QUINDI $L_i \in DEC$

QUINDI $\forall i \ L_i \in DEC$

$$\bullet E_{TM} = \{ \langle M \rangle : M \text{ è una TM ed } L(M) = \emptyset \}$$

PREMESSA: SAPPIAMO CHE $A_{TM} = \{ \langle M, w \rangle : M \text{ è una TM, } w \in \Sigma^* \text{ e } M \text{ accetta } w \}$ NON È UN

PROBLEMA DECIDIBILE,

$A_{TM} \notin DEC$

PER VERIFICARE CHE E_{TM} SIA INDECIDIBILE CREO UNA TM T, SUPPONIAMO PER ASSURDO CHE E_{TM} SIA DECIDIBILE QUINDI ESISTE H DECISORE PER E_{TM} .

T PRENDE IN INPUT $\langle M, w \rangle$

CREA UNA NUOVA TM $\bar{M}$

SE $x \neq w$ T RIFIUTA

SE $x = w$

SIMULA x SU M

SE M ACCETTA

$\bar{M}$ ACCETTA

ALTRIMENTI RIFIUTA

SIMULA H SU $\bar{M}$

SE H ACCETTA

T ACCETTA

ALTRIMENTI T RIFIUTA

T È UN DECISORE PER A_{TM} MA QUESTO NON È POSSIBILE QUINDI H NON È DECISORE PER E_{TM}

2 Calcolabilità

10 Points

- Descrivere formalmente una macchina di Turing che riconosca il linguaggio delle stringhe binarie contenenti lo stesso numero di 0 e di 1. Dimostrare la correttezza della soluzione proposta.
- Sia L un linguaggio arbitrario. Dimostrare che L è decidibile se e solo se L è Turing-riconoscibile e coTuring-riconoscibile.

POSSIAMO COSTRUIRE UNA GRAMMATICA CFG E POI ESEGUIRE QUESTO IN UNA TM M

DEFINIAMO G' CFG

$$G' = (\{S, Z, U\}, \{0, 1\}, R, \{S\})$$

$$R: \begin{cases} S \rightarrow ZU \mid \epsilon \\ Z \rightarrow ZZU \mid 0 \\ U \rightarrow UUZ \mid 1 \end{cases}$$

IL PROBLEMA $A_{CFG} = \{ \langle G, w \rangle : G \text{ CFG}, w \in \Sigma^* \wedge G \text{ produce } w \}$

È TURING DECIDIBILE PERCHÉ POSSIAMO CREARE UN DECISORE M CHE, DATO IN INPUT $\langle G, w \rangle$ CREA UN GRAFO, DOVE I NODI SONO TUTTE LE STRINGHE DI Σ^* . DUE NODI q, p SONO LEGATI DA UNA FRECCIA SE q DERIVA p . L'UNICO NODO MARCATO ALL'INIZIO È IL NODO CORRISPONDENTE ALLA STRINGA w . INIZIO UN CICLO, MARCO AD OGNI ITERAZIONE, FINCHÉ È POSSIBILE TUTTI I NODI CHE HANNO UN ARCO USCENTE VERSO UN NODO GIÀ MARCATO. A FINE CICLO, SE HO MARCATO "S" (NODO CORRISPONDENTE) ALLORA M ACCETTA ALTRIMENTI RIFIUTA.

PER RISOLVERE IL QUESITO BASTA QUINDI CREARE UNA TM M' CHE SIMULA M CON INPUT $\langle G, w \rangle$ SE M ACCETTA M' ACCETTA ALTRIMENTI RIFIUTA.

• $LE_{DEC} \Leftrightarrow LE_{REC} \wedge LE_{COREC}$

($\Leftarrow$)

$LE_{REC} \wedge LE_{COREC}$ POSSIAMO CREARE UNA TM M CHE ESEGUE IN PARALLELO M_1, M_2 t.e. $L(M_1), L(M_2)$ SU UN INPUT w , DATO CHE $L \cup \bar{L} = \Sigma^*$ UNA DELLE DUE TM M_1 E M_2 AD UN CERTO PUNTO ACCETTERÀ QUINDI LE_{DEC}

($\Rightarrow$)

$LE_{DEC} \Rightarrow LE_{REC}$, SE UN LINGUAGGIO È DECIDIBILE È PURE RICONOSCIBILE.

$$LE_{DEC} \Rightarrow \bar{L} \in DEC \Rightarrow \bar{L} \in REC \Rightarrow (LE_{REC}) \wedge (LE_{COREC})$$

BASTA NEGARE NELLA MACCHINA DI TURING AL POSTO DI ACCETTARE

• Si consideri il linguaggio W_{TM} delle stringhe della forma $\langle M, w, a \rangle$, dove M è una TM, w è una stringa appartenente all'alfabeto di input di M , ed a è un simbolo appartenente all'alfabeto di nastro di M , tale che M scrive a sul nastro quando è eseguita su input w . Mostrare che W_{TM} è indecidibile.

• Dimostrare che un linguaggio è Turing-riconoscibile se e solo se esiste una TM multi-nastro che lo riconosce.

PUNTO 2

UN LINGUAGGIO È TURING-RICONOSCIBILE QUANDO ESISTE UNA TM CHE RICONOSCE QUEL LINGUAGGIO, DATO CHE TM E NTM HANNO LA STESSA POTENZA ESPRESSIVA

UN LINGUAGGIO È TURING-RICONOSCIBILE QUANDO ESISTE UNA NTM CHE RICONOSCE QUEL LINGUAGGIO.

DIMOSTRIAMO CHE $TM = NTM$

SUPPONIAMO DI AVERE UNA NTM

CREIAMO UNA TM, LE INFORMAZIONI CONTENUTE NEGLI n NASTRI POSSONO ESSERE SCRITTE SU NASTRO SINGOLO, L'ALFABETO DELLA TM DEVE AVERE PER OGNI CARATTERE x IL CARATTERE $\hat{x}$ CHE INDICA DOVE LEGGE LA TESTINA SU UN NASTRO SIMULATO. OGNI VOLTA CHE LA NTM COMPIE UN PASSO LEGGE SU m NASTRI CON m TESTINE E AGISCE DI CONSEGUENZA, LA NTM INVECE VEDRÀ DOVE SI TROVANO I CARATTERI "PUNTATI" E AGIRÀ DI CONSEGUENZA

□

PUNTO 1

$W_{TM} = \{ \langle M, w, a \rangle : TM M, w \in \Sigma^*, a \in \Gamma^*, \begin{matrix} a \text{ scritto su nastro} \\ \text{se leggo } w \end{matrix} \}$

SUPPONIAMO CHE W_{TM} SIA DECIDIBILE QUINDI ESISTE UN DECISORE D CHE DECIDE W_{TM}

CREIAMO LA TM A CHE PRENDE IN INPUT $\langle M, w \rangle$ COSTRUISCE TM M' CHE

STAMPA a SE M ACCETTA w
ALTRIMENTI NO

ESEGUE $D(\langle M', w, a \rangle)$

SE D ACCETTA A ACCETTA

ALTRIMENTI RIFIUTA.

A È DECISORE DI A_{TM} IN QUESTO MODO

MA A_{TM} È INDECIDIBILE QUINDI NON È POSSIBILE, QUINDI D NON È DECISORE

2 Parte Seconda

10 Points

- Uno stato di una macchina di Turing è detto inutile se la macchina non entra mai in quello stato per ogni possibile input. Considerare il problema di determinare se una macchina di Turing ha uno stato inutile. Dimostrare che il linguaggio associato a questo problema non è decidibile.
- Dimostrare che esistono linguaggi che non sono decidibili e neppure Turing-riconoscibili.

$$I_{TM} = \{ \langle M \rangle : M \text{ è una TM e ha uno stato inutile} \}$$

SENZA ENTRARE NEL DETTAGLIO POSSIAMO AFFERHARE CHE, SE I_{TM} È DECIDIBILE $\Rightarrow \exists D$ TM DECISORE PER I_{TM}

- SUPPONIAMO ORA DI AVERE

T TM CHE FA QUESTO:

RICEVE INPUT $\langle M, w \rangle$

CREA M'

- CONFRONTA w CON x , SE $w \neq x$ RIFIUTA ALTREMENTI
- SIMULA M SU INPUT w
- SE M ACCETTA M' ACCETTA ALTREMENTI RIFIUTA

SIMULA $D(\langle M' \rangle)$

SE D ACCETTA T RIFIUTA
SE D RIFIUTA T

2 Parte Seconda

10 Points

- Sia $DECIDABLE_{TM}$ il linguaggio che consiste di tutte le stringhe $\langle M \rangle$ tali che M è una macchina di Turing ed $L(M)$ è decidibile. Mostrare che $DECIDABLE_{TM}$ è indecidibile.
- Dimostrare che per ogni macchina di Turing non-deterministica ne esiste una deterministica equivalente.

$$DECIDABLE_{TM} = \{ \langle M \rangle : L(M) \in DEC \}$$

SUPPONIAMO CHE SIA DECIDIBILE, QUINDI ESISTERA' UN DECISORE D

POSSIAMO CREARE T , CHE PRENDE IN INPUT $\langle M, w \rangle$

- CREA M' CON INPUT x
SE $w \neq x$ RIFIUTA
SE $w == x$
SIMULA M SU INPUT (w)
SE M ACCETTA, M' ACCETTA
SE M RIFIUTA, M' RIFIUTA
- SIMULA D SU INPUT $\langle M' \rangle$
SE D ACCETTA, T ACCETTA
SE D RIFIUTA, T RIFIUTA.

NO

2 Parte Seconda

10 Points

- Sia $DECIDABLE_{TM}$ il linguaggio che consiste di tutte le stringhe $\langle M \rangle$ tali che M è una macchina di Turing ed $L(M)$ è decidibile. Mostrare che $DECIDABLE_{TM}$ è indecidibile.
- Dimostrare che per ogni macchina di Turing non-deterministica ne esiste una deterministica equivalente.

SUPPONIAMO CHE $DECIDABLE_{TM}$ SIA DECIDIBILE, ALLOR ESISTE D TM DECISORE.

POSSIAMO CREARE LA TMS CHE PRENDE IN INPUT $\langle M, w \rangle$

CREA M' CHE FUNZIONA COSÌ:

SIMULA M SU INPUT w

SE M ACCETTA, M' ACCETTA

ALTRIMENTI ACCETTA

SIMULA D CON INPUT M'

SE D ACCETTA S RIFIUTA

ALTRIMENTI S ACCETTA.

$$\langle M, w \rangle \in A_{TM} \Leftrightarrow S \text{ ACCETTA } \langle M, w \rangle$$

MA A_{TM} È INDECIDIBILE PERTANTO $DECIDABLE_{TM}$ È INDECIDIBILE

2 Parte Seconda

10 Points

- Sia $HALTS_ON_ALL_{TM}$ il linguaggio che consiste di tutte le stringhe della forma $\langle M \rangle$ tali che M è una macchina di Turing che termina sempre per ogni possibile scelta dell'input. Mostrare che $HALTS_ON_ALL_{TM}$ è indecidibile.
- Dimostrare che il linguaggio $EQ_{TM} = \{ \langle M_1, M_2 \rangle : L(M_1) = L(M_2) \}$, dove M_1 ed M_2 sono macchine di Turing, è indecidibile.

$$HOA_{TM} = \{ \langle M \rangle : M \text{ è una TM, } M \text{ termina sempre} \}$$

SUPPONIAMO PER ASSURDO CHE HOA_{TM} SIA DECIDIBILE, ESISTE QUINDI D DECISORE TM PER IL PROBLEMA.

POSSIAMO CREARE S , CON INPUT $\langle M, w \rangle$, MACCHINA D I TURING CHE:

CREA M' CHE:

SIMULA M SU w

SE ACCETTA ACCETTA

ALTRIMENTI ACCETTA

SIMULA D SU M'

SE D ACCETTA, S RIFIUTA

ALTRIMENTI S ACCETTA

D ACCETTA M' SE $L(M')$ E' DECIDIBILE
 SE $L(M') = A_{TM}$ D RIFIUTA E S ACCETTA
 QUINDI S E' UN DECISORE PER A_{TM} MA QUESTO E'
 IMPOSSIBILE DATO CHE A_{TM} E' INDECIDIBILE QUINDI
 D NON E' DECISORE PERTANTO $HO_{A_{TM}}$ E' INDECIDIBILE

Dimostrare che il linguaggio $EQ_{TM} = \{\langle M_1, M_2 \rangle : L(M_1) = L(M_2)\}$, dove M_1 ed M_2 sono macchine di Turing, è indecidibile.

DI MOSTRAZIONE:

POSSIAMO CALCOLARE $\Delta(L(M_1), L(M_2))$

DOVE $\Delta(L(M_1), L(M_2))$, CHIAMANDO $L(M_1) = L_1$ E $L(M_2) = L_2$

$$\Delta(L_1, L_2) = (\bar{L}_1 \cap L_2) \cup (L_1 \cap \bar{L}_2)$$

SE QUESTO Δ VALE 0 I DUE LINGUAGGI SONO UGUALI.

POSSIAMO CREARE LA TM S CHE PRENDE 2 LINGUAGGI ($L(M_1)$ E $L(M_2)$)

PRODUCE $\Delta(L(M_1), L(M_2))$

CONTROLLA CHE QUESTO Δ ... (CHE E' UN LINGUAGGIO) SIA NULLO

QUINDI SE $\Delta(L(M_1), L(M_2)) \in E_{TM}$

SE SI S ACCETTA

ALTRIMENTI RIFIUTA

$$\langle M_1, M_2 \rangle \in S \iff \Delta(L(M_1), L(M_2)) \in E_{TM}$$

MA E_{TM} E' INDECIDIBILE QUINDI S E' INDECIDIBILE

SAPPIAMO CHE E_{TM} E' INDECIDIBILE QUINDI EQ_{TM} E' INDECIDIBILE

NO.